Multimodal Sentiment Analysis via Multi-Layer Feature Fusion & Multi-Task Learning

A Re-implementation and Extension of Cai et al. (2025)

Gautam Gandhi Manas Inamdar Parth Tokekar Soham Jahagirdar

Introduction to NLP (S26) — IIIT Hyderabad

Team: TokeNization

March 2026

What We'll Cover

1. The Paper & The Problem
2. Architecture: What the Paper Proposes
3. What the Paper Specifies vs. Our Assumptions
4. Phase 1: First Implementation
5. Phase 2: Improved Architecture (v2)
6. Results & Comparison
7. Verdict & Next Steps

The Paper We're Implementing

Reference

Cai et al., "*Multimodal sentiment analysis based on multi-layer feature fusion and multi-task learning*,"
Scientific Reports (2025), 15:2126.

Our Goal

Re-implement this architecture from scratch in PyTorch and understand every design choice.

Core idea in one sentence:

Use **text + audio + video** together to predict how positive or negative someone feels — fusing all three using attention, trained with multiple simultaneous tasks.

Key claim: Better fusion + multi-task training gives
MAE = 0.728 Acc-2 = 86.6% on CMU-MOSI.

Dataset

CMU-MOSI
2,199 video clips
sentiment: -3 to $+3$
1,283 train / 214 val / 686 test

The Problem: Why is This Hard?

What is Multimodal Sentiment Analysis?

- Given a video clip of someone speaking, predict their sentiment score
- Use **text** (what they say), **audio** (how they say it), and **video** (what their face shows)

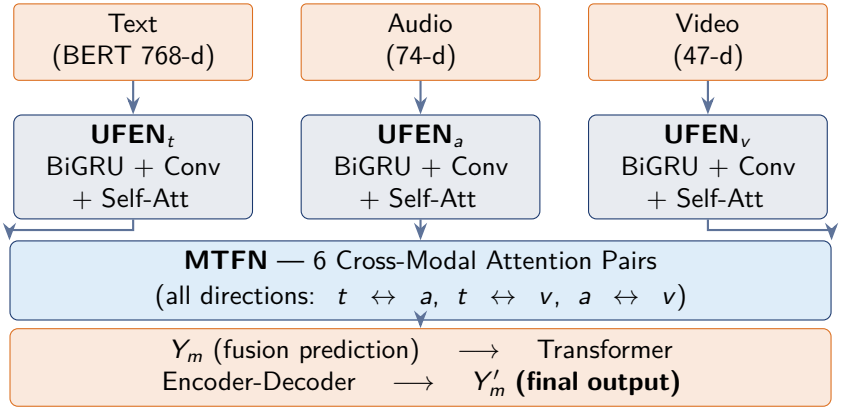
Why not just use text?

In "I *absolutely loved* it" with a *flat voice* and *blank face* — the words say positive but the person means the opposite. Text alone gets fooled by sarcasm and irony.

Three Core Challenges

1. **Different data types:** Text is 768-dim word vectors; audio is 74 acoustic measurements; video is 47 facial action scores. They live in very different spaces.
2. **Conflicting signals:** One modality might say positive while another says negative. Which do you trust?
3. **Text dominates:** Text alone gives $\sim$73% accuracy. Audio and video add only a little — but they help on the hard cases.

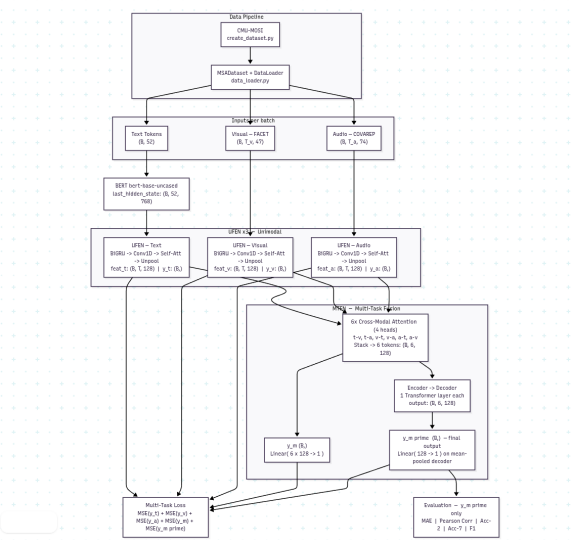
Overall Architecture: Two Stages



Each UFEN also outputs Y_t, Y_a, Y_v (training only)

Stage 1 (UFEN): Extract good features from each modality independently. **Stage 2 (MTFN):** Combine all three modalities using attention.

Complete System Pipeline (v2)



Stage 1 — UFEN: Learning Features per Modality

Why do we need UFEN? Raw input features are noisy and have no temporal context understood by the network yet. UFEN builds a compact, context-aware representation for each modality.

UFEN Pipeline (same structure for all 3 modalities):

1. **Bi-GRU** — reads the sequence forward *and* backward to capture context (e.g., "not bad at all" — the word "bad" gets corrected by what comes after it)
2. **N parallel Conv1D branches** — detect local patterns of different widths (e.g., 1-word vs. 5-word windows)
3. **Self-Attention gate** — let each time step decide how much other time steps matter for

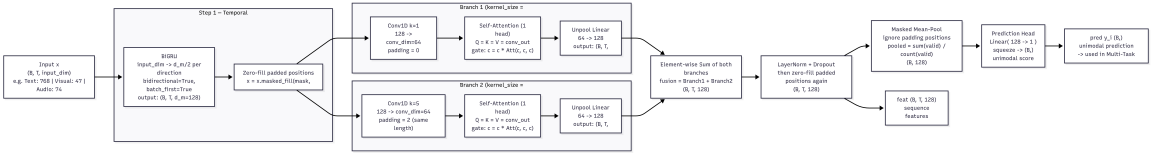
Why Bi-GRU?

A regular GRU only sees past words. Bidirectional also sees future words, which helps with word-order-sensitive sentiment.

Why Conv branches?

Short utterances (5–14 words average) benefit from local pattern detection at multiple scales — a single kernel misses either local or wider context.

UFEN: Forward Pass in Detail



Stage 2 — MTFN: Fusing the Three Modalities

Cross-Modal Attention (6 directed pairs):

Each pair lets one modality *ask questions* about another:

- Text asks: *what does the audio confirm/contradict?*
- Audio asks: *what emotion does the face confirm?*
- All 6 directions: $t \to a$, $t \to v$, $a \to t$, $a \to v$, $v \to t$, $v \to a$

Why cross-attention and not just concatenation?

Concatenation treats all features equally. Attention *selectively* pulls in only the relevant parts of the other modality — crucial when modalities conflict.

Transformer Encoder-Decoder:

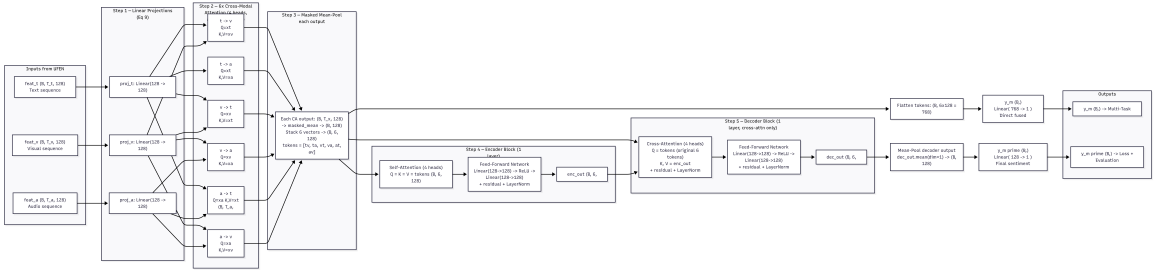
- The 6 attention outputs are mean-pooled $\to$ 6 tokens
- Encoder: refines the 6 tokens using self-attention
- Decoder: produces the final representation $\to Y'_m$

Multi-Task Learning (5 losses!)

$$\text{Loss} = \lambda_m \mathcal{L}_{m'} + \lambda_f \mathcal{L}_m + \lambda_t \mathcal{L}_t + \lambda_a \mathcal{L}_a + \lambda_v \mathcal{L}_v$$

Training on all 5 heads forces each UFEN to stay useful on its own — prevents the model from ignoring audio/video.

MTFN: Forward Pass in Detail



What the Paper Specifies Clearly

Directly from Table 2 or the ablation study:

Parameter	Value	Source
Self-att heads	1	Table 2
Cross-att heads	4	Table 2
Att dropout	0.2	Table 2
Dropout	0.1	Table 2
Starting LR	5e-3	Table 2
Batch size	32	Table 2
Epochs	30	Table 2
Conv branches (N)	2	Ablation

What is *not* specified

- Hidden dimension d_m
- Conv kernel sizes
- Conv filter count
- FFN size in encoder-decoder
- Whether BERT is **frozen** or **fine-tuned**
- How to handle variable-length sequences
- LR to use for BERT
- Whether to use an LR schedule

These gaps forced us to make design decisions — and they turned out to be the most important decisions in the whole project.

Our Key Assumptions

Assumption 1 — BERT's Role

Paper calls it "feature extractor" (Experiments) but UFEN is described uniformly for all modalities (Methods). **Ambiguous:** frozen or fine-tuned?

Assumption 2 — $d_m = 128$

With 4 cross-attention heads, $d_m = 128$ gives 32 dims/head — a standard choice. Never stated explicitly in the paper.

Assumption 3 — Conv kernels

Two parallel branches with different kernels: $k = 1$ (per-word) and $k = 5$ (local context), since utterances average 5–14 words.

Assumption 4 — Softmax → Linear output

Paper writes $Y = \text{softmax}(Wx + b)$, but labels are continuous (-3 to $+3$). We use `Linear(d,1)` — softmax would break regression.

Assumption 5 — BERT LR separation

Paper says LR = 5e-3. Fine-tuning BERT at 5e-3 destroys pre-trained weights. Solution: BERT at 2e-5, rest at 5e-3.

Assumption 6 — Mean-pooling for fusion

Cross-attention outputs have variable length. We mean-pool each output over time → one vector per pair → stack to 6 tokens.

Phase 1 — First Implementation (`main` branch)

A careful, conservative reading of the paper

Key design choices:

- **BERT frozen** — text features pre-extracted once and saved
- **Sequential conv branches** — each layer feeds the next (kernel = 3 for all)
- **Shared cross-attention weights** — all 6 pairs share Q/K/V per modality (literal read of Eq. 9)
- Fixed $T = 30$; no padding masks
- Single LR = 5e-3 for all parameters
- Early stop patience = 8 on validation loss

Architecture snapshot

d_m	64
d_e (cross-att)	64
GRU hidden	32 / direction
Visual dim	35 (FACET 4.2)
Total params	**442,501**

Results

MAE ↓	0.981
Pearson Corr ↑	0.618
Acc-7 ↑	34.3%
Acc-2 (neg/pos) ↑	77.7%

Paper target: MAE 0.728, Acc-2 86.6%

Why Phase 1's Numbers Were Lower

After the mid-submission we investigated *why* the gap existed.

Root cause analysis:

- Frozen BERT: model **cannot adapt text representations** — static features, even for ambiguous sentiment words
- Early stop patience = 8 with noisy val-loss stopped training too early
- Single LR = 5e-3 with no schedule ⇒ noisy convergence
- Zero-padding without masks pollutes attention and mean-pool

The biggest insight

Frozen BERT ⇒ only **442K trainable parameters** from 1,283 training examples.

BERT representations are fixed — the model cannot learn that "sick" means *great* in slang, or "fine" in a flat voice means *unhappy*.

Fine-tuning BERT is likely what the paper intended, despite the "feature extractor" wording.

After More Research — What We Changed for v2

After the mid-submission, we continued experimenting and built a revised implementation (`ufen-mtfn-v2`).

Dimension	Phase 1 (main)	Phase 2 (v2)
BERT	Frozen, pre-extracted	Fine-tuned live
Total parameters	442K	110.8M
Conv branches	Sequential, same kernel	Parallel, $k = 1$ and $k = 5$
Cross-attention	Shared Q/K/V (6 pairs)	6 independent modules
Padding masks	Not used	Full mask propagation
LR strategy	Single 5e-3	Dual: 5e-3 + 2e-5 (BERT)
LR schedule	None	Cosine + BERT warmup
d_m	64	128
Visual dim	35	47

Key Difference 1 — BERT: Frozen vs. Fine-tuned

Phase 1 (frozen BERT):

- BERT runs once, saves 768-dim vectors to disk
- Vectors never change during training
- Model learns on top of fixed representations
- Only 442K parameters are trainable

Phase 2 (fine-tuned BERT):

- BERT runs inside every forward pass
- Weights update at small LR (2e-5) to preserve pre-trained knowledge
- Text representations adapt continuously to predict sentiment
- 110M parameters become trainable

Why does fine-tuning help so much?

BERT is pre-trained on generic language. Sentiment has domain-specific patterns (sarcasm, hedging, intensifiers) the generic representations don't capture well.

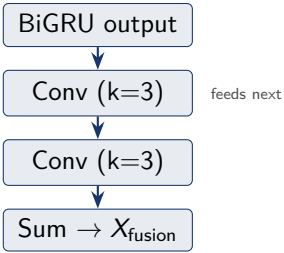
Fine-tuning lets BERT specialise for sentiment on top of its broad knowledge.

The BERT warmup trick

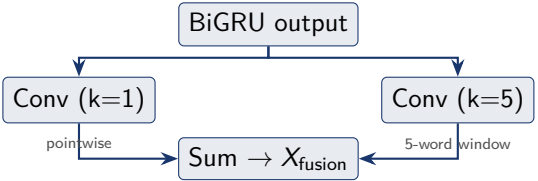
BERT is frozen for the first 5 epochs so the rest of the model stabilises, *then* BERT is unfrozen. This prevents chaotic gradients from destroying BERT at startup.

Key Difference 2 — Conv Branches: Sequential vs. Parallel

Phase 1 — Sequential:



Phase 2 — Parallel:



Both branches read the *same* BiGRU output.
Different kernels give multi-scale features.

Each conv layer reads *the previous layer's output*.
Both branches use the same kernel size.

Why parallel is better: The paper says "*N parallel* branches" — parallel directly matches that wording. Different kernel sizes let the model capture both per-word features ($k = 1$) and short-phrase features ($k = 5$) simultaneously.

Key Difference 3 — Training Strategy

Phase 1 — Simple training:

- Single LR = 5e-3 for *all* parameters
- No learning rate schedule
- Early stop on total validation loss (sum of 5 sub-task MSE losses)
- Patience = 8 epochs

Problem: Validation total loss is dominated by the 3 unimodal sub-tasks. The model can stop early even if the main combined output is still improving.

Phase 2 — Richer training:

- **Dual learning rates:**
  - 5e-3 for non-BERT parameters (fresh, needs to learn fast)
  - 2e-5 for BERT (pre-trained, needs gentle updates)
- **Cosine LR schedule** — smoothly decays LR over 50 epochs instead of keeping it constant
- **BERT warmup** — BERT is frozen for 5 epochs, then unfrozen gradually
- Early stop on **dev MAE directly** — what we actually care about
- Patience = 15 epochs

Results: Phase 1 vs Phase 2 vs Paper Target

Metric	Phase 1	Phase 2	Improvement	Paper Target
MAE ↓	0.981	**0.812**	−0.169	0.728
Pearson Corr ↑	0.618	**0.745**	+0.127	0.792
Acc-7 ↑	34.3%	**43.0%**	+8.7pp	46.7%
Acc-2 (neg/pos) ↑	77.7%	**80.3%**	+2.6pp	86.6%
Trainable params	442K	110.8M	—	—

Best run (P3-2 config)

50 epochs total, early stopped at epoch 42
Best dev MAE = 0.7434 (close to paper's 0.728)
Test MAE = 0.812 — small dev/test gap of 0.069

What's still missing vs. paper

The remaining gap likely comes from: norm differences in their data preprocessing, exact FACET/COVAREP feature versions, and possible differences in multi-task loss weights.

What Made the Biggest Difference

We ran 25+ experiments across 3 phases to isolate each factor:

Change	MAE gain	Impact
BERT fine-tuning	~ -0.12	Very high
Cosine LR + warmup	~ -0.03	Medium
Parallel conv + larger k	~ -0.02	Moderate
Masked mean-pool	~ -0.01	Minor
Early stop on MAE	~ -0.01	Minor

Key lesson

Whether to freeze or fine-tune BERT accounts for **the majority of the gap** between Phase 1 and Phase 2.

Many papers say "BERT as feature extractor" but actually fine-tune it in code. We had to determine the right interpretation experimentally.

Also important

Proper LR scheduling and metric-targeted early stopping had a clear effect on final numbers.

Which Design Choices Were Right?

Decision	Phase 1	Phase 2	Verdict
BERT role	Frozen	Fine-tuned	v2 correct
Conv design	Sequential, $k = 3$	Parallel, $k = 1, 5$	v2 closer
Cross-att weights	Shared	Independent	Both defensible
Padding masks	Not used	Full masking	v2 rigorous
Early stop metric	Total val loss	Dev MAE	v2 better
LR schedule	None	Cosine + warmup	Clearly helpful
Z-score norm	Applied	Not applied	Phase 1 better

Phase 1's Z-score normalisation is **the one thing Phase 1 did better** — cleaning extreme COVAREP artifact values ($\approx -3.4 \times 10^{38}$) that v2 skips, relying on model adaptation instead.

Next Steps

Near-term experiments on CMU-MOSI:

- Try LR = 2e-3 for non-BERT params to reduce the learning rate asymmetry (current: 250:1 ratio, BERT vs. non-BERT)
- Add Z-score normalisation from Phase 1 into v2 preprocessing
- Explore SWA (Stochastic Weight Averaging) — a technique that averages model weights over the last few epochs to improve generalisation; already implemented in Phase 1's code

Project Phase 2 — MELD dataset:

- Adapt the UFEN+MTFN architecture for **emotion classification** (7 classes: Anger, Disgust, Sadness, Joy, Neutral, Surprise, Fear)
- Replace regression head with softmax classification
- Training objective: Cross-Entropy instead of MSE
- Evaluate with weighted F1 (class imbalance in MELD)
- Qualitative analysis: visualise attention weights on sarcastic clips to see if model uses facial cues correctly

Summary

What we implemented:

- Full PyTorch re-implementation of UFEN + MTFN from Cai et al. (2025)
- Two versions with careful documentation of every design choice and assumption
- 25+ systematic experiments across 3 phases to understand what matters most

Best result achieved (Phase 2):

MAE	**0.812** (paper: 0.728)
Pearson Corr	**0.745** (paper: 0.792)
Acc-2	**80.3%** (paper: 86.6%)
Acc-7	**43.0%** (paper: 46.7%)

Key takeaways

1. **BERT fine-tuning is the most important decision** — ambiguous paper wording led to Phase 1 keeping BERT frozen, which was the root cause of the large performance gap.
2. **Architecture details matter** — parallel convolutions with multi-scale kernels outperform sequential same-kernel branches.
3. **Training setup matters** — proper LR scheduling and metric-targeted early stopping are often as important as the architecture itself.

Thank You!

Gautam Gandhi Manas Inamdar
Parth Tokekar Soham Jahagirdar

Introduction to NLP (S26) — Team TokeNization

Cai et al. (2025). "Multimodal sentiment analysis based on multi-layer feature fusion and multi-task learning." *Scientific Reports*, 15:2126.